ThreatLattice

A Security Knowledge Graph for Weakness-to-Technique Attack Chain Analysis

DBMS Experiential Learning, Level 3 Project
Deliverable 1: Requirements 1 to 5
Section 5A, CSE
Abhigyan Dutta (PES2UG24CS019) and Akshansh Agrawal (PES2UG24CS045)

---

1. Project Title

Title: ThreatLattice, A Security Knowledge Graph for Weakness-to-Technique Attack Chain Analysis.

Problem statement

Security teams rely on four separate reference catalogues to make sense of a single attack. CWE lists the kinds of weaknesses that appear in software. CVE lists the actual vulnerabilities found in real products. CAPEC lists the patterns attackers use to exploit those weaknesses. ATT&CK lists the techniques attackers have really been observed using. Each catalogue is maintained by a different group and lives on its own website, so joining them up is a manual job.

These catalogues are linked in practice but stored apart. When an analyst finds a vulnerability in a scan, the useful next question is what an attacker could actually do with it. Answering that means moving from the CVE to its weakness, then to the attack patterns that exploit that weakness, then to the real techniques those patterns enable. Doing this by hand across four sites is slow and easy to get wrong.

ThreatLattice loads all four catalogues into one system and connects them, so that walk becomes a single query. It is not a create, read, update, and delete application. The value of the project is in the relationships between the catalogues and the ability to move across them, not in storing plain records.

2. Team Formation

The project is built by two people. Each of us owns part of both the database and the application, so neither of us is only a database person or only an application person.

Abhigyan Dutta (PES2UG24CS019) handles the graph side and the security side. This covers the Neo4j data model and its constraints, loading the CAPEC and ATT&CK data along with the mapping edges between catalogues, and writing the graph queries that walk the attack chain. On the application side he handles login, the two user roles, and the page that lets a user explore a chain starting from a CVE.

Akshansh Agrawal (PES2UG24CS045) handles the relational side and the analytics side. This covers the MySQL schema and all of the table definitions, loading the CVE and CWE data, and building the views, trigger, stored function, and full-text search. On the application side he handles the dashboard that shows severity breakdowns and lets a user search CVE descriptions.

Some parts are shared. We review each other's database design, write the documentation together, connect the application to both databases, and test the finished system as a pair. Our commit history on GitHub records who wrote what.

3. Database Requirement

The project uses two database paradigms, which matches Option 1 in the requirements: a relational database and a NoSQL database. We use MySQL for the relational part, which is mandatory, and Neo4j, a graph database, for the NoSQL part.

We did not choose two databases just to meet the rule. The data falls into two kinds that are best handled in different ways.

The first kind is record data with many attributes that we filter, sort, and group. A CVE has a severity score, a severity band, a publish date, and a set of affected products. Questions like show every critical vulnerability published this year, sorted by score, are ordinary relational queries. MySQL handles these well using indexes and constraints.

The second kind is relationship data that we follow across catalogues. The main question the project answers, starting from this vulnerability which real techniques could follow, is a path of several hops: CVE to weakness, weakness to attack pattern, attack pattern to technique. In a relational database this turns into a chain of joins that grows longer and slower with every hop. In a graph database the same thing is one pattern that the database follows directly, and it is easier to read and to extend. Neo4j is built for this.

So each database does the job it is suited to. The relational store holds the attribute-heavy records, and the graph store holds the connections between catalogues. A few links, such as the one between a CVE and its weakness, sit in both stores, because the relational side needs it for filtering and the graph side needs it as the first step of the chain. This is a deliberate design choice, not accidental duplication.

4. Database Design

The relational part was designed as an ER diagram first and then turned into tables. The diagram is shown in Figure 1. The design has three groups of tables. The catalogue tables hold the entries themselves. The relationship tables connect them. The application tables support login and saved work.

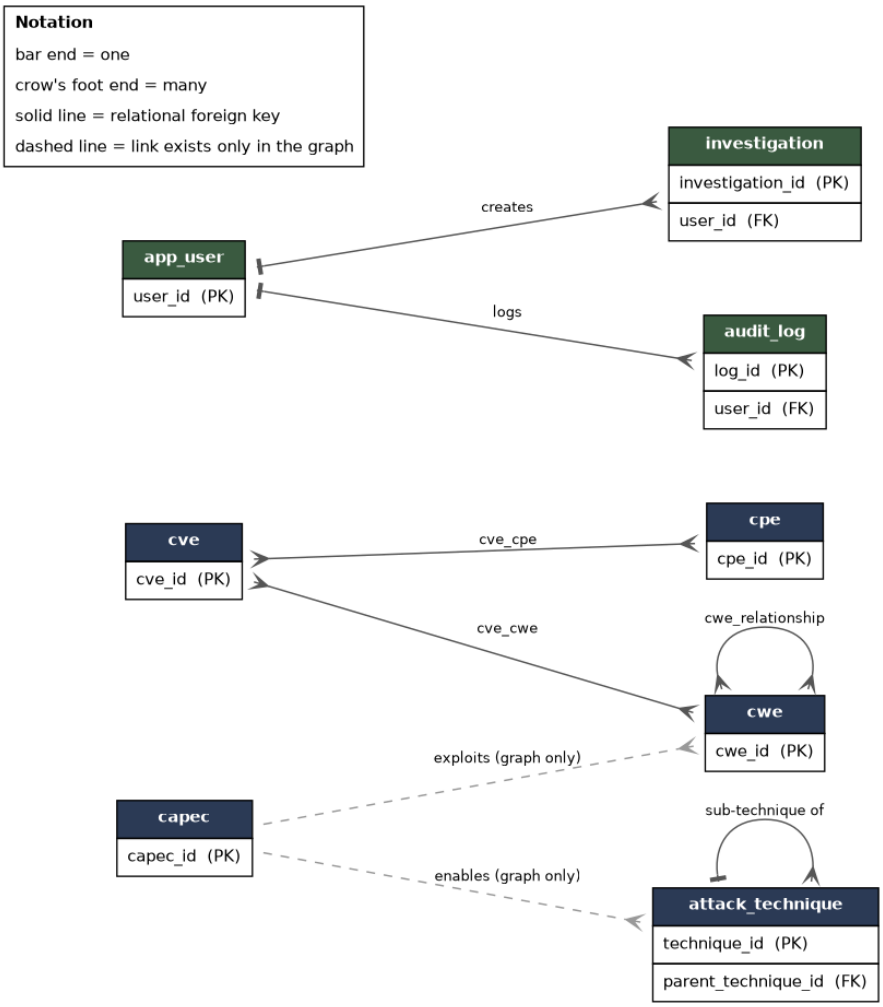


Figure 1. Simplified ER diagram for the relational part, drawn in crow's foot notation.

Each many-to-many relationship is shown as a single line labelled with the junction table that implements it: cve_cwe, cve_cpe, and cwe_relationship. The dashed lines from capec are links that exist only in the graph database, which is why capec has no relational foreign keys of its own.

Relational schema

The five catalogue tables are cwe, cve, capec, attack_technique, and cpe. Their primary keys are the natural catalogue identifiers, for example CWE-89 or CVE-2021-44228, except for cpe which uses a generated numeric key. The three relationship tables are cve_cwe and cve_cpe, which are many to many junctions, and cwe_relationship, which is a self referencing junction for the weakness hierarchy. The attack_technique table also references itself through parent_technique_id to hold sub-techniques. The three application tables are app_user, investigation, and audit_log. The data dictionary below lists each table with its columns, keys, and constraints.

cwe

Column	Type	Key	Constraints / Notes
cwe_id	VARCHAR(12)	PK	Format CWE-<number>
name	VARCHAR(255)		Not null
abstraction	VARCHAR(20)		
status	VARCHAR(20)		
description	TEXT		

cve

Column	Type	Key	Constraints / Notes
cve_id	VARCHAR(20)	PK	Format CVE-YYYY-<number>
description	TEXT		Not null, full-text indexed
published_date	DATE		Not null
last_modified_date	DATE		
cvss_base_score	DECIMAL(3,1)		Between 0.0 and 10.0
cvss_severity	ENUM		LOW, MEDIUM, HIGH, CRITICAL
cvss_vector	VARCHAR(128)		

capec

Column	Type	Key	Constraints / Notes
capec_id	VARCHAR(12)	PK	Format CAPEC-<number>
name	VARCHAR(255)		Not null
abstraction	VARCHAR(20)		
likelihood_of_attack	VARCHAR(20)		
typical_severity	ENUM		LOW, MEDIUM, HIGH, CRITICAL
description	TEXT		

attack_technique

Column	Type	Key	Constraints / Notes
technique_id	VARCHAR(12)	PK	ATT&CK technique id, e.g. T1190
name	VARCHAR(255)		Not null
tactic	VARCHAR(64)		
platform	VARCHAR(128)		
is_subtechnique	BOOLEAN		Default false
parent_technique_id	VARCHAR(12)	FK	Self reference to attack_technique
description	TEXT		

cpe

Column	Type	Key	Constraints / Notes
cpe_id	BIGINT	PK	Auto increment
vendor	VARCHAR(128)		Not null
product	VARCHAR(128)		Not null

Column	Type	Key	Constraints / Notes
version	VARCHAR(64)		Unique with vendor and product

app_user

Column	Type	Key	Constraints / Notes
user_id	BIGINT	PK	Auto increment
username	VARCHAR(50)		Unique, not null
email	VARCHAR(255)		Unique, checked for valid form
password_hash	VARCHAR(255)		Not null, stores a hash only
role	ENUM		analyst or admin, default analyst
created_at	TIMESTAMP		Default current time

investigation

Column	Type	Key	Constraints / Notes
investigation_id	BIGINT	PK	Auto increment
user_id	BIGINT	FK	References app_user
title	VARCHAR(200)		Not null
start_node_type	VARCHAR(20)		In CVE, CWE, CAPEC, TECHNIQUE
start_node_id	VARCHAR(20)		Not null
notes	TEXT		
created_at	TIMESTAMP		Default current time

audit_log

Column	Type	Key	Constraints / Notes
log_id	BIGINT	PK	Auto increment
user_id	BIGINT	FK	References app_user
action	VARCHAR(20)		Not null
entity	VARCHAR(40)		Not null
entity_id	VARCHAR(40)		
ts	TIMESTAMP		Default current time

Relationship (junction) tables

Table	Columns and keys	Purpose and constraints
cve_cwe	cve_id, cwe_id (both PK and FK)	Links a vulnerability to its weakness types
cve_cpe	cve_id, cpe_id (both PK and FK)	Links a vulnerability to affected products
cwe_relationship	parent_cwe_id, child_cwe_id (both PK and FK)	Weakness hierarchy. Check stops a row linking a weakness to itself

Graph data model

The graph database does not use an ER diagram, because Neo4j is not relational. It is described instead as nodes and relationships.

Nodes. There are four node types, each with a unique id: CVE, CWE, CAPEC, and Technique. Each node also stores a small set of properties. For example a CVE node keeps its score and severity, and a Technique node keeps its name and tactic.

Relationships. A CVE is an INSTANCE_OF a CWE. A CAPEC EXPLOITS a CWE. A CAPEC ENABLES a Technique. A CWE can be a CHILD_OF another CWE. A Technique can be a SUBTECHNIQUE_OF another Technique. The two mappings from CAPEC, to CWE and to Technique, live only in the graph, since they exist only to be traversed.

Indexes. Each node type has a uniqueness constraint on its id. In Neo4j a uniqueness constraint also builds an index, so looking up a node by its id is fast.

Embeddings. There are no embeddings in this project. We are not doing semantic or vector search, so vectors and similarity metrics do not apply here.

Retrieval strategy. Retrieval starts by looking up a node by its exact id, then follows the relationships out from it to build the chain. For the shortest connection between two points, for example from a weakness to a technique, we use Neo4j shortest path search.

5. SQL Implementation

The relational schema is created with the DDL below. It runs on MySQL 8.0. It covers table creation, an alter statement that adds a full-text index, primary and foreign keys, check and unique constraints, secondary indexes, and a view. The drop statements at the end are kept as comments and are used only to reset the schema.

```
-- schema.sql  (MySQL 8.0)

-- Node tables
CREATE TABLE cwe (
    cwe_id      VARCHAR(12) PRIMARY KEY,
    name        VARCHAR(255) NOT NULL,
    abstraction VARCHAR(20),
    status      VARCHAR(20),
    description TEXT,
    CONSTRAINT chk_cwe_id CHECK (cwe_id REGEXP '^CWE-[0-9]+$')
);

CREATE TABLE capec (
    capec_id             VARCHAR(12) PRIMARY KEY,
    name                 VARCHAR(255) NOT NULL,
    abstraction          VARCHAR(20),
    likelihood_of_attack VARCHAR(20),
    typical_severity     ENUM('LOW','MEDIUM','HIGH','CRITICAL'),
    description          TEXT,
    CONSTRAINT chk_capec_id CHECK (capec_id REGEXP '^CAPEC-[0-9]+$')
);

CREATE TABLE attack_technique (
    technique_id        VARCHAR(12) PRIMARY KEY,
    name                VARCHAR(255) NOT NULL,
    tactic              VARCHAR(64),
    platform            VARCHAR(128),
    is_subtechnique     BOOLEAN NOT NULL DEFAULT FALSE,
    parent_technique_id VARCHAR(12),
    description         TEXT,
```

```
    CONSTRAINT fk_attack_parent FOREIGN KEY (parent_technique_id)
        REFERENCES attack_technique(technique_id) ON DELETE SET NULL
);

CREATE TABLE cpe (
    cpe_id  BIGINT AUTO_INCREMENT PRIMARY KEY,
    vendor  VARCHAR(128) NOT NULL,
    product VARCHAR(128) NOT NULL,
    version VARCHAR(64),
    CONSTRAINT uq_cpe UNIQUE (vendor, product, version)
);

CREATE TABLE cve (
    cve_id             VARCHAR(20) PRIMARY KEY,
    description        TEXT NOT NULL,
    published_date     DATE NOT NULL,
    last_modified_date DATE,
    cvss_base_score    DECIMAL(3,1),
    cvss_severity      ENUM('LOW','MEDIUM','HIGH','CRITICAL'),
    cvss_vector        VARCHAR(128),
    CONSTRAINT chk_cve_id     CHECK (cve_id REGEXP '^CVE-[0-9]{4}-[0-9]+$'),
    CONSTRAINT chk_cvss_range CHECK (cvss_base_score BETWEEN 0.0 AND 10.0)
);

-- Relationship tables
CREATE TABLE cve_cwe (
    cve_id VARCHAR(20) NOT NULL,
    cwe_id VARCHAR(12) NOT NULL,
    PRIMARY KEY (cve_id, cwe_id),
    FOREIGN KEY (cve_id) REFERENCES cve(cve_id) ON DELETE CASCADE,
    FOREIGN KEY (cwe_id) REFERENCES cwe(cwe_id) ON DELETE CASCADE
);

CREATE TABLE cve_cpe (
    cve_id VARCHAR(20) NOT NULL,
    cpe_id BIGINT NOT NULL,
    PRIMARY KEY (cve_id, cpe_id),
    FOREIGN KEY (cve_id) REFERENCES cve(cve_id) ON DELETE CASCADE,
    FOREIGN KEY (cpe_id) REFERENCES cpe(cpe_id) ON DELETE CASCADE
);

CREATE TABLE cwe_relationship (
    parent_cwe_id     VARCHAR(12) NOT NULL,
    child_cwe_id      VARCHAR(12) NOT NULL,
    relationship_type VARCHAR(20) NOT NULL DEFAULT 'ChildOf',
    PRIMARY KEY (parent_cwe_id, child_cwe_id),
    FOREIGN KEY (parent_cwe_id) REFERENCES cwe(cwe_id) ON DELETE CASCADE,
    FOREIGN KEY (child_cwe_id)  REFERENCES cwe(cwe_id) ON DELETE CASCADE,
    CONSTRAINT chk_no_self_ref CHECK (parent_cwe_id <> child_cwe_id)
);

-- Application tables
CREATE TABLE app_user (
    user_id       BIGINT AUTO_INCREMENT PRIMARY KEY,
    username      VARCHAR(50)  NOT NULL UNIQUE,
    email         VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role          ENUM('analyst','admin') NOT NULL DEFAULT 'analyst',
    created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_email CHECK (email REGEXP '^[^@]+@[^@]+\\.[^@]+$')
);

CREATE TABLE investigation (
    investigation_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id          BIGINT NOT NULL,
    title            VARCHAR(200) NOT NULL,
```

```
    start_node_type  VARCHAR(20)  NOT NULL,
    start_node_id    VARCHAR(20)  NOT NULL,
    notes            TEXT,
    created_at       TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES app_user(user_id) ON DELETE CASCADE,
    CONSTRAINT chk_start_type CHECK (start_node_type IN ('CVE','CWE','CAPEC','TECHNIQUE'))
);

CREATE TABLE audit_log (
    log_id    BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id   BIGINT,
    action    VARCHAR(20) NOT NULL,
    entity    VARCHAR(40) NOT NULL,
    entity_id VARCHAR(40),
    ts        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES app_user(user_id) ON DELETE SET NULL
);

-- ALTER: add a full-text index on CVE descriptions
ALTER TABLE cve ADD FULLTEXT INDEX idx_cve_fts (description);

-- Indexes
CREATE INDEX idx_cve_published ON cve(published_date);
CREATE INDEX idx_cve_score     ON cve(cvss_base_score);
CREATE INDEX idx_cve_severity  ON cve(cvss_severity);
CREATE INDEX idx_cvecwe_cwe    ON cve_cwe(cwe_id);
CREATE INDEX idx_cvecpe_cpe    ON cve_cpe(cpe_id);

-- View: critical CVEs joined to their weakness
CREATE VIEW v_critical_cve AS
SELECT c.cve_id, c.cvss_base_score, w.cwe_id, w.name AS cwe_name
FROM   cve c
JOIN   cve_cwe cc ON cc.cve_id = c.cve_id
JOIN   cwe w      ON w.cwe_id  = cc.cwe_id
WHERE  c.cvss_severity = 'CRITICAL';

-- Teardown (DROP demonstration), run only to reset the schema
-- DROP VIEW IF EXISTS v_critical_cve;
-- SET FOREIGN_KEY_CHECKS = 0;
-- DROP TABLE IF EXISTS audit_log, investigation, app_user,
--   cwe_relationship, cve_cpe, cve_cwe, cve, cpe,
--   attack_technique, capec, cwe;
-- SET FOREIGN_KEY_CHECKS = 1;
```

The graph database has its own equivalent of DDL. The statements below create a uniqueness constraint on each node type, which also builds the index used for lookups during traversal. These run in the Neo4j query editor.

```
CREATE CONSTRAINT cwe_id   IF NOT EXISTS FOR (n:CWE)       REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT cve_id   IF NOT EXISTS FOR (n:CVE)       REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT capec_id IF NOT EXISTS FOR (n:CAPEC)     REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT tech_id  IF NOT EXISTS FOR (n:Technique) REQUIRE n.id IS UNIQUE;

SHOW CONSTRAINTS;
```

Together these statements demonstrate the required data definition work: creating structure, altering it, dropping it, primary and foreign keys, constraints, indexes, and a view on the relational side, with matching constraints and indexes on the graph side.

Contributions

The table below records which of us prepared each part of this deliverable, in line with our roles.

Part	Prepared by
Project title and problem statement	Both
Team formation	Both
Database requirement and justification	Both
ER diagram, relational schema, data dictionary	Akshansh Agrawal
Graph data model	Abhigyan Dutta
MySQL DDL	Akshansh Agrawal
Neo4j constraints	Abhigyan Dutta